

LangChain Interview Q&A

Pure framework-level questions with real-world examples - no personal project references

1. LangChain Basics

Q: What is LangChain and why was it created?

LangChain is an open-source framework for building applications powered by LLMs. It provides standard abstractions for prompts, chains, memory, retrieval, and agents so developers don't have to write the same LLM orchestration logic from scratch for every app. It exists because raw LLM API calls are stateless and don't naturally handle multi-step reasoning, external data, or tool use.

Real-world example: An airline wants a chatbot that checks flight status, answers FAQs, and can rebook a ticket. Instead of writing custom glue code to combine an API call, a knowledge base search, and the LLM response, LangChain gives ready-made components (tools, retrievers, agents) to wire this together fast.

Q: What are the main building blocks of LangChain?

Models (LLMs/ChatModels), Prompts (templates), Chains (sequences of calls), Memory (state across turns), Indexes/Retrievers (for connecting external data), and Agents (dynamic tool-using reasoning loops). Everything else in the framework is built on top of these six ideas.

Q: What's the difference between an LLM and a ChatModel in LangChain?

An LLM takes a plain string in and returns a plain string out (text completion style). A ChatModel is designed around a list of messages (system, human, AI) and returns a message - matching how modern chat-tuned models like GPT-4 or Gemini actually work. Almost all production LangChain apps use ChatModels now since most providers are chat-first.

Q: What is a PromptTemplate and why not just use an f-string?

PromptTemplate lets you define a prompt with named variables, validate required inputs, and reuse/version it independently of code logic. It also integrates with LangChain's tracing (so you can see exactly what prompt was sent) and supports partial variables, few-shot example injection, and output format instructions in a structured way that plain string formatting doesn't give you.

Real-world example: A hotel booking assistant might have one PromptTemplate for 'answer FAQ' and another for 'summarize guest complaint' - both reusable across different guest inputs without duplicating code.

Q: What is an Output Parser? Give an example of when you'd need one.

An Output Parser converts the LLM's raw text response into a structured format your application code can use directly - like a Python dict, a Pydantic object, or a list. Without it, you'd have to write fragile regex to pull fields out of free text.

Real-world example: A resume-screening app asks the LLM to extract candidate name, years of experience, and skills. Using a PydanticOutputParser (or JsonOutputParser) with a defined schema forces the LLM's output into clean JSON fields that get stored directly in a database, instead of parsing free-form paragraphs with regex.

2. Chains & LCEL

Q: What is LCEL and what problem does it solve?

LCEL (LangChain Expression Language) lets you compose components with the pipe operator: `prompt | model | output_parser`. It replaces older class-based chains (LLMChain, SimpleSequentialChain) with a declarative syntax where every component is a 'Runnable'. This automatically gives you streaming, batching, async support, and retry/fallback logic without extra code, and each step becomes individually

traceable.

Real-world example: A customer support summarizer: `prompt | chat_model | StrOutputParser()` - this one line handles a ticket's raw text, sends it to the LLM with a formatted prompt, and returns a clean string summary, and it streams token-by-token to the UI automatically.

Q: What's the difference between SequentialChain and RunnableSequence (LCEL)?

SequentialChain (legacy) is an explicit Python class where you define input/output keys manually and chain multiple LLMChains together. RunnableSequence (LCEL, created via the `|` operator) does the same thing but is simpler to write, composes more flexibly (can nest parallel and conditional runnables), and integrates cleanly with LangSmith tracing and streaming.

Q: What is a Router Chain and when would you use one?

A Router Chain examines the input and decides which of several downstream chains/prompts should handle it, instead of always running the same fixed chain. It's useful when a single app needs to serve very different types of queries with different specialized prompts.

Real-world example: A single company chatbot handling both 'billing questions' and 'technical support questions' - a router chain looks at the incoming message and routes billing queries to a chain with billing-specific context, and technical queries to a different chain with product documentation context.

Q: What is RunnableParallel and RunnablePassthrough used for?

RunnableParallel runs multiple Runnables at the same time on the same input and returns their combined outputs as a dict - useful when a prompt needs several pieces of information assembled together. RunnablePassthrough simply forwards the original input unchanged into a later step, which is commonly needed in RAG chains where you need both the retrieved context and the original question available to the final prompt.

Real-world example: In a RAG chain: `RunnableParallel({'context': retriever, 'question': RunnablePassthrough()})` - this fetches relevant documents (context) while also passing the original user question through untouched, so both can be inserted into the final prompt template together.

Q: What is Map-Reduce in the context of LangChain chains?

It's a summarization/processing strategy for documents too large to fit in one context window: the 'map' step summarizes each chunk individually and in parallel, then the 'reduce' step combines those chunk-level summaries into one final summary. LangChain has a built-in `load_summarize_chain` with `chain_type='map_reduce'` for this.

Real-world example: Summarizing a 200-page legal contract - each section is summarized separately (map), then all section summaries are combined into one executive summary (reduce), since the whole contract can't fit in a single prompt.

3. RAG & Retrieval

Q: Explain the RAG (Retrieval-Augmented Generation) pattern in LangChain.

RAG combines a retrieval step with a generation step: relevant chunks of external data are fetched from a vector store based on the user's query, and those chunks are inserted into the prompt as context before the LLM generates its final answer. This lets the LLM answer using information it was never trained on, and reduces hallucination since answers are grounded in retrieved text.

Real-world example: An internal company chatbot that answers HR policy questions - it retrieves the relevant paragraph from the HR handbook (which the LLM has never seen during training) and generates an answer using only that retrieved text.

Q: What's the difference between a Document Loader, a Text Splitter, and a Retriever?

A Document Loader brings raw data (PDF, website, database rows, etc.) into LangChain's standard Document format. A Text Splitter breaks large documents into smaller chunks that fit within embedding and

context-window limits. A Retriever is the query-time component that takes a user question and returns the most relevant chunks from a vector store. They represent three different stages: ingestion, preprocessing, and query-time lookup.

Q: What is chunk overlap and why does it matter?

Chunk overlap means consecutive chunks share a small amount of text at their boundary, so a sentence or idea that would otherwise be split awkwardly across two chunks still appears intact in at least one of them. Too little overlap risks losing context at chunk edges; too much overlap wastes storage and increases redundant retrieval.

Real-world example: A user manual chunked without overlap might split 'To reset the device, hold the power button for 10 seconds' across two chunks, losing the instruction's full meaning in either chunk alone. With overlap, the full instruction stays together in at least one chunk.

Q: What's the difference between similarity search and Maximal Marginal Relevance (MMR) in retrieval?

Plain similarity search returns the top-k chunks closest to the query vector, which can return several near-duplicate chunks if the source data is repetitive. MMR re-ranks results to balance relevance with diversity, so the returned chunks cover different aspects of the topic instead of saying the same thing five different ways.

Real-world example: Searching a product FAQ database where three FAQ entries are nearly identical - MMR ensures the retriever returns one of those plus other, more diverse relevant entries, instead of returning all three near-duplicates.

Q: What is a hybrid retriever and why would you use one?

A hybrid retriever (e.g. LangChain's EnsembleRetriever) combines keyword-based search (like BM25) with vector-based semantic search, then merges/re-ranks results. This catches both semantic matches (similar meaning, different wording) and exact-term matches (product codes, names, IDs) that pure embedding search sometimes misses.

Real-world example: A legal document search tool: a user searching for 'Section 12.3(a)' needs exact keyword matching (BM25 handles this well), while a user asking 'what happens if the tenant breaks the lease early' needs semantic search - a hybrid retriever handles both query types well.

Q: How does a self-querying retriever work?

It uses the LLM to parse a natural-language query into both a semantic search string and structured metadata filters, then applies both to the vector store. This lets users ask questions that combine content search with filtering, without writing structured queries themselves.

Real-world example: A user asks 'find me reviews from 2023 that mention battery life' for a product review database - the self-querying retriever separates this into a semantic search for 'battery life' plus a metadata filter for year=2023, and the vector store applies both.

4. Memory

Q: Why can't you just pass the entire chat history to the LLM every time?

Every LLM has a maximum context window, and sending the full, ever-growing conversation history eventually exceeds it - and even before that limit, it increases cost and latency on every call since you're re-sending the same tokens repeatedly. Memory modules manage this by summarizing, windowing, or selectively retrieving relevant past turns instead of sending everything.

Q: Compare ConversationBufferMemory, ConversationBufferWindowMemory, and ConversationSummaryMemory.

ConversationBufferMemory stores the full raw conversation - simple but grows unbounded.

ConversationBufferWindowMemory keeps only the last k turns, bounding size but losing older context

entirely. ConversationSummaryMemory uses the LLM itself to continuously compress older turns into a running summary, keeping the essence of long conversations within a small token budget.

Real-world example: A therapy-style chatbot with sessions lasting an hour would benefit from ConversationSummaryMemory so it remembers what was discussed 40 minutes ago without blowing the context window, whereas a quick 3-question customer support bot is fine with ConversationBufferMemory.

Q: What is entity memory and when is it useful?

Entity memory tracks facts about specific entities mentioned in a conversation (people, places, things) rather than the whole conversation flow, updating a structured record of what's known about each entity as the conversation progresses.

Real-world example: A sales assistant chatbot that remembers 'the client's company is a 50-person startup, based in Pune, budget under 5 lakhs' as distinct entity facts, so later questions in the same conversation can reference these without re-asking or re-reading the whole transcript.

5. Agents & Tools

Q: What is an agent in LangChain, and how is it different from a chain?

A chain follows a fixed, developer-defined sequence of steps every time. An agent uses the LLM itself to decide, at runtime, which tool to call, in what order, and when it has enough information to give a final answer - it reasons dynamically rather than following a hardcoded path.

Real-world example: A travel-booking assistant: a chain would always run 'search flights -> then always search hotels' in that fixed order even if the user only asked about flights. An agent would reason 'the user only asked about flights, so I'll only call the flight-search tool' and skip the hotel tool entirely.

Q: Explain the ReAct pattern used by LangChain agents.

ReAct stands for Reason + Act. The agent alternates between a 'Thought' (reasoning about what to do next), an 'Action' (calling a tool), and an 'Observation' (the tool's result), looping until it decides it has enough information to produce a final answer. This makes the agent's decision-making process interpretable, since you can see its reasoning trace.

Real-world example: Asked 'what's the weather in the city where the Eiffel Tower is', a ReAct agent reasons: Thought - I need to know which city the Eiffel Tower is in -> Action - search('Eiffel Tower location') -> Observation - Paris -> Thought - now get weather -> Action - weather_tool('Paris') -> final answer.

Q: What is a Tool in LangChain, and what makes a good tool description?

A Tool is a function (or another chain) wrapped with a name and a natural-language description, exposed to an agent so the LLM can decide when to call it. The description matters a lot because the LLM chooses tools based purely on reading that description - a vague description leads to wrong tool selection.

Real-world example: A tool named 'get_order_status' should have a description like 'Use this to look up the current shipping status of a customer's order given an order ID' - specific enough that the agent doesn't confuse it with a similarly-named 'get_order_history' tool.

Q: What's the risk of using agents in production, and how do you mitigate it?

Agents are non-deterministic and can loop indefinitely, call the wrong tool, or rack up unexpected API costs since the LLM is making the decisions rather than fixed code. Mitigations: set a max_iterations limit, add timeout/fallback responses, log every tool call for auditing, and restrict the toolset to only what's strictly necessary for the task.

Real-world example: A finance chatbot with tool access to 'transfer_funds' should have strict guardrails - explicit confirmation steps, restricted amounts, and audit logging - since an agent misinterpreting an ambiguous request could otherwise trigger a real transaction.

Q: What's the difference between a ReAct agent and an OpenAI Functions / Tool-calling agent?

A ReAct agent relies on prompting the LLM to output text in a specific Thought/Action/Observation format, which the framework then parses - this can be brittle if the model doesn't follow the format exactly. A tool-calling agent uses the model provider's native function/tool-calling capability (structured JSON tool calls built into the model itself), which is more reliable and doesn't depend on fragile text parsing.

6. Embeddings & Vector Stores

Q: What is an embedding, in plain terms?

An embedding is a fixed-length list of numbers (a vector) that represents the meaning of a piece of text, generated by an embedding model. Texts with similar meaning end up with vectors that are mathematically close together, which is what makes semantic search possible.

Real-world example: The sentences 'I want to cancel my subscription' and 'How do I stop my membership' would have very similar embedding vectors even though they share almost no exact words, letting a search system match them as semantically related.

Q: How do you choose between different vector stores (FAISS, Chroma, Pinecone, Weaviate)?

Considerations: scale (millions of vectors need a managed, distributed store like Pinecone/Weaviate; smaller datasets work fine locally), whether you need metadata filtering, whether you want a managed cloud service versus self-hosting, and cost. FAISS/Chroma are great for local development and smaller-scale apps; Pinecone/Weaviate/Milvus are built for production-scale, multi-tenant, filterable search.

Q: What is cosine similarity and why is it commonly used for vector search?

Cosine similarity measures the angle between two vectors rather than their magnitude, giving a score (usually -1 to 1) of how similar their direction is. It's preferred for text embeddings because it captures semantic similarity regardless of how 'long' or intense the text is, unlike distance metrics that could be skewed by vector magnitude.

7. Comparisons & Tricky Conceptual Questions

Q: LangChain vs LlamaIndex - how would you differentiate them?

Both help build LLM applications over external data, but LlamaIndex historically focused more deeply on data indexing and retrieval strategies for RAG specifically, while LangChain is broader - covering agents, tools, memory, and general orchestration alongside RAG. In practice, teams often use LlamaIndex for advanced indexing and LangChain for the surrounding orchestration/agent logic, or just pick whichever has better support for their specific need.

Q: What is LangServe and why would you use it?

LangServe helps deploy LangChain Runnables/chains as production REST APIs quickly, auto-generating OpenAPI docs and handling streaming responses out of the box, so you don't have to hand-write FastAPI boilerplate around every chain.

Q: What is LangSmith and how is it different from LangChain?

LangChain is the framework for building the application; LangSmith is Anthropic-independent tooling (by the LangChain team) for tracing, debugging, evaluating, and monitoring those applications in production - showing you every prompt, retrieved document, token count, and latency per step of a run.

Q: Can you build a LangChain app without using any chains or agents at all?

Yes - you can use just the Model and Prompt components directly for a single-turn Q&A, without memory, retrieval, or agents. Chains/agents only become necessary once you need multi-step logic, external data, or dynamic tool use.

Q: What are the limitations of LangChain as a framework?

It can add abstraction overhead for very simple use cases where a direct API call would be simpler and more transparent. Debugging deeply nested chains without LangSmith can be hard since errors get buried across abstraction layers. The framework also evolves fast, so APIs and best practices (like the shift from legacy chains to LCEL) change frequently, requiring code updates to stay current.

Q: How would you handle a situation where the LLM's tool call has invalid or malformed arguments?

Wrap tool execution in validation (e.g. Pydantic schema for expected arguments) and catch errors gracefully - return a clear error message back to the agent as the 'Observation' so it can retry with corrected arguments, rather than letting the whole chain crash.

Real-world example: An agent calls a 'book_flight' tool with a malformed date string - instead of crashing, the tool returns 'Error: date must be in YYYY-MM-DD format' as an observation, and the agent retries with the corrected format.

Tip: Interviewers often ask 'can you give me an example' right after a definition - the real-world examples above are there so you have one ready without freezing up.